

Temas Tratados en el Trabajo Práctico 4

- Representación del Conocimiento y Razonamiento Lógico.
- Estrategias de resolución de hipótesis: Encadenamiento hacia Adelante, Encadenamiento hacia Atrás y Resolución por Contradicción.
- Representación basada en circuitos.

Ejercicios Teóricos

1. ¿Qué es una inferencia?

Una **inferencia** es el proceso lógico mediante el cual, a partir de una o varias premisas conocidas en la base de conocimiento (KB), se deriva una nueva fórmula (conclusión). En lógica proposicional: si de KB se puede derivar φ mediante reglas de inferencia (por ejemplo modus ponens, resolución), decimos que $KB \models \varphi$ (φ es consecuencia lógica de KB).

2. ¿Cómo se verifica que un modelo se infiere de la base de conocimientos?

Hay dos enfoques relacionados:

- **Semántico:** verificar que φ es consecuencia de KB significa comprobar que en todas las interpretaciones (modelos) que satisfacen KB también se satisface φ . Esto es la definición semántica: $KB \models \varphi$.
- **Sintáctico:** usar un algoritmo de inferencia (por ejemplo resolución en CNF, encadenamiento hacia adelante/atrás para cláusulas Horn) que pruebe φ a partir de las reglas de KB. Si el algoritmo es sonido (no deriva falsos) y completo (puede derivar todo lo que semánticamente se sigue), entonces la prueba sintáctica confirma la inferencia.

3. Observe la siguiente base de conocimiento:

$R1 : b \wedge c \rightarrow a$

$R2 : d \wedge e \rightarrow b$

$R3 : g \wedge e \rightarrow b$

$R4 : e \rightarrow c$

$R5 : d$

$R6 : e$

$$R7 : a \wedge g \rightarrow f$$

3.1 ¿Cómo se puede probar que $a = \text{True}$ a través del encadenamiento hacia adelante? Este método solamente usa reglas ya incorporadas a la base de conocimiento para inferir la hipótesis, ¿qué propiedad debe tener el algoritmo para asegurar que esta inferencia sea posible?

3.2 ¿Cómo se puede probar que $a = \text{True}$ a través del encadenamiento hacia atrás? Este método asigna un valor de verdad a la hipótesis y deriva las sentencias de la base de conocimiento, ¿qué propiedad debe tener el algoritmo para asegurar que esta derivación sea posible?

3.3 Expresar la base de conocimiento en su Forma Normal Conjuntiva. A continuación, demuestre por contradicción que $a = \text{True}$.

3.1. Encadenamiento hacia adelante (forward chaining) para probar $a = \text{True}$

Procedimiento (encadenamiento hacia adelante): partimos de los hechos conocidos y aplicamos reglas cuyo antecedente esté satisfecho, añadiendo las conclusiones hasta que aparezca a o no haya más reglas aplicables.

Aplicación concreta:

1. Hechos iniciales: d (R5), e (R6).
2. Con R4: de $e \rightarrow c$. Ahora sabemos c .
3. Con R2: de $d \wedge e \rightarrow b$. Dado que tenemos d y e , inferimos b .
4. Con R1: de $b \wedge c \rightarrow a$. Ahora que tenemos b y c , inferimos a .

Por tanto, con encadenamiento hacia adelante llegamos a: $a = \text{True}$.

Propiedad requerida: Para asegurar que esta inferencia sea posible por encadenamiento hacia adelante, el algoritmo debe ser sonido y completo respecto a la clase de fórmulas (aquí la KB es un conjunto de cláusulas Horn). En particular: forward chaining es completo para cláusulas Horn.

3.2. Encadenamiento hacia atrás (backward chaining) para probar $a = \text{True}$

Procedimiento (encadenamiento hacia atrás): comenzamos con la meta a y buscamos reglas que concluyan a . Si una regla tiene $b \wedge c \rightarrow a$, entonces el procedimiento intenta probar b y c recursivamente. Si alguna submeta es un hecho conocido, se acepta; si no, se busca derivarla.

Aplicación concreta:

1. Queremos a . Solo R1 tiene a en el consecuente: requerimos probar b y c .
2. Probar c : usar R4 ($e \rightarrow c$). Entonces probamos e . e está hecho (R6) $\rightarrow$ OK.

3. Probar b: usar R2 ($d \wedge e \rightarrow b$). Necesitamos d y e. d es hecho (R5), e es hecho (R6) $\rightarrow$ OK.
4. Ambas submetas se prueban, por tanto a se prueba.

Propiedad requerida: backward chaining es completo para cláusulas Horn si se emplea una estrategia de búsqueda que evite ciclos (p. ej. memoización) y explore las reglas necesarias. Es decir: completitud y prevención de loops.

3.3. Forma Normal Conjuntiva (CNF) y demostración por contradicción de $a = \text{True}$

Cada implicación $p \wedge q \rightarrow r$ se convierte en la cláusula $(\neg p \vee \neg q \vee r)$. Transformamos las reglas/hechos a CNF (cláusulas):

1. R1: $\neg b \vee \neg c \vee a$
2. R2: $\neg d \vee \neg e \vee b$
3. R3: $\neg g \vee \neg e \vee b$
4. R4: $\neg e \vee c$
5. R5: d
6. R6: e
7. R7: $\neg a \vee \neg g \vee f$

Demostración por contradicción (resolución): asumimos $\neg a$ y probamos contradicción.

Cláusulas iniciales:

- (1) $\neg b \vee \neg c \vee a$
- (2) $\neg d \vee \neg e \vee b$
- (3) $\neg g \vee \neg e \vee b$
- (4) $\neg e \vee c$
- (5) d
- (6) e
- (7) $\neg a \vee \neg g \vee f$
- (8) $\neg a$ (supuesto para contradicción)

Resolución paso a paso:

- (2) con (5=d) $\Rightarrow$ resolvente (9): $\neg e \vee b$
- (9) con (6=e) $\Rightarrow$ resolvente (10): b
- (4) con (6=e) $\Rightarrow$ resolvente (11): c
- (1) con (10=b) $\Rightarrow$ resolvente (12): $\neg c \vee a$
- (12) con (11=c) $\Rightarrow$ resolvente (13): a
- (13) a con (8) $\neg a \Rightarrow$ cláusula vacía $\rightarrow$ contradicción.

Por tanto, la asunción $\neg a$ conduce a contradicción; concluimos que $a = \text{True}$.

Observación: no se necesitaron R3 ni R7 en esta demostración.

4. Diseñe con lógica proposicional basada en circuitos las proposiciones *OrientadoDerecha* y *Agente ubicado en la casilla [1,2]* para el mundo de wumpus de 4x4. Dibuje el circuito correspondiente.
5. El nonograma es un juego en el cual se posee un tablero en blanco y cada fila y columna presenta información sobre la longitud de un bloque en dicha fila/columna. Además, la leyenda puede indicar más de un número, indicando esto que existen varios bloques de las longitudes mostradas por la leyenda y en el mismo orden, separados por al menos un espacio vacío.

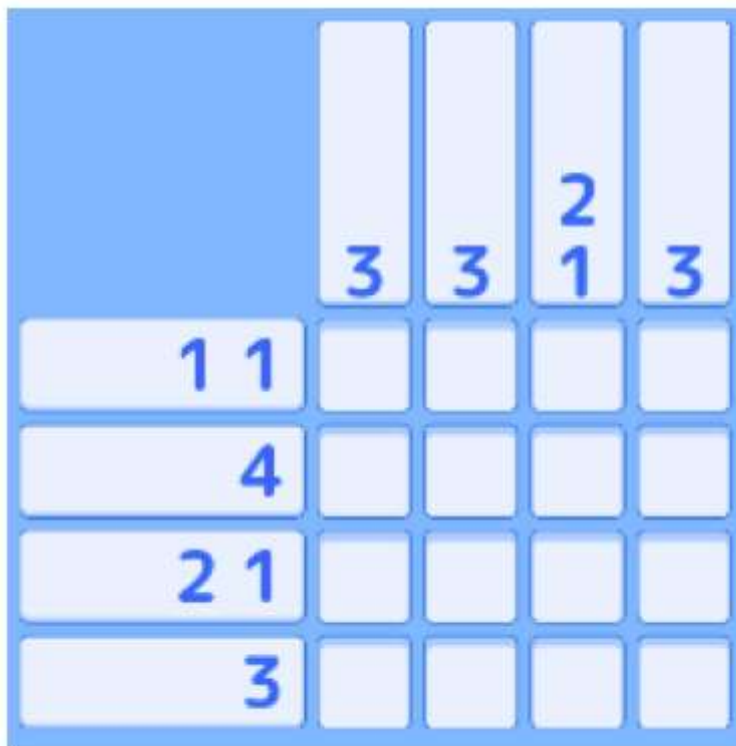
Resuelva el nonograma de la imagen de abajo escribiendo en primer lugar cada regla que puede incorporarse a la base de conocimientos inicial e incorporando cada inferencia que realice.

```
In [1]: import requests
        from PIL import Image
        from io import BytesIO
        import matplotlib.pyplot as plt

        # URL directa de Google Drive
        url = "https://drive.google.com/uc?export=view&id=1SKiXvrI_TX-U4sbw60TYSRmaNYyFi

        # Descargar la imagen
        response = requests.get(url)
        img = Image.open(BytesIO(response.content))

        # Mostrar la imagen
        plt.imshow(img)
        plt.axis('off') # Ocultar ejes
        plt.show()
```



1. Pistas (enunciado)

- Filas (de arriba hacia abajo):

F1: [1,1]

F2: [4]

F3: [2,1]

F4: [3]

- Columnas (izquierda a derecha):

C1: [3]

C2: [3]

C3: [2,1]

C4: [3]

2. Reglas básicas del nonograma

Reglas usadas:

- Cada número indica la longitud de un bloque de casillas negras en esa fila/columna, en el orden dado.
- Los bloques sucesivos están separados por al menos una casilla blanca.
- Si la pista ocupa toda la fila/columna (ej. [4] en fila de 4), todas las casillas son blancas.

- Las casillas no marcadas explícitamente como negras se consideran potencialmente blancas hasta la demostración.

3. Reglas lógicas

- R1: $\text{Cas}(1,1) \wedge \text{Cas}(1,3) \vee \text{Cas}(1,2) \wedge \text{Cas}(1,4) \vee \text{Cas}(1,1) \wedge \text{Cas}(1,4)$
- R2: $\text{Cas}(2,1) \wedge \text{Cas}(2,2) \wedge \text{Cas}(2,3) \wedge \text{Cas}(2,4)$
- R3: $\text{Cas}(3,1) \wedge \text{Cas}(3,2) \wedge \text{Cas}(3,4) \vee \text{Cas}(3,1) \wedge \text{Cas}(3,3) \wedge \text{Cas}(3,4)$
- R4: $\text{Cas}(4,1) \wedge \text{Cas}(4,2) \wedge \text{Cas}(4,3) \vee \text{Cas}(4,2) \wedge \text{Cas}(4,3) \wedge \text{Cas}(4,4)$
- R5: $\text{Cas}(1,1) \wedge \text{Cas}(2,1) \wedge \text{Cas}(3,1) \vee \text{Cas}(2,1) \wedge \text{Cas}(3,1) \wedge \text{Cas}(4,1)$
- R6: $\text{Cas}(1,2) \wedge \text{Cas}(2,2) \wedge \text{Cas}(3,2) \vee \text{Cas}(2,2) \wedge \text{Cas}(3,2) \wedge \text{Cas}(4,2)$
- R7: $\text{Cas}(1,3) \wedge \text{Cas}(2,3) \wedge \text{Cas}(4,3) \vee \text{Cas}(1,3) \wedge \text{Cas}(3,3) \wedge \text{Cas}(4,3)$
- R8: $\text{Cas}(1,4) \wedge \text{Cas}(2,4) \wedge \text{Cas}(3,4) \vee \text{Cas}(2,4) \wedge \text{Cas}(3,4) \wedge \text{Cas}(4,4)$

Ahora podemos comenzar a inferir, y empezaremos por R2 ya que solo tiene una opción:

R2.1: Cas (2,1)

R2.2: Cas (2,2)

R2.3: Cas (2,3)

R2.4: Cas (2,4)

Fila 1: ☐ ☐ ☐ ☐

Fila 2: ☒ ☒ ☒ ☒

Fila 3: ☐ ☐ ☐ ☐

Fila 4: ☐ ☐ ☐ ☐

La notación será ☒ para las casillas inferidas y ☐ para las que no, el tablero empezaría como un 4x4 negro.

Ahora nos movemos a R7 ya que de las dos opciones que hay solo una cumple con lo ya inferido para Cas (2,3):

R7.1: Cas (1,3)

R7.2: Cas (4,3)

Fila 1: ☐ ☐ ☒ ☐

Fila 2: ☒ ☒ ☒ ☒

Fila 3: ☐ ☐ ☐ ☐

Fila 4: ☐ ☐ ☒ ☐

Podemos ir ahora a R1 donde solo tenemos una posibilidad:

R1.1: Cas (1,1)

Fila 1: ■ □ ■ □

Fila 2: ■ ■ ■ ■

Fila 3: □ □ □ □

Fila 4: □ □ ■ □

Nos queda una sola posibilidad para R5:

R5.1: Cas (3,1)

Fila 1: ■ □ ■ □

Fila 2: ■ ■ ■ ■

Fila 3: ■ □ □ □

Fila 4: □ □ ■ □

Ya cumplimos con R7 por lo que Cas (3,3) no es posible, de esta manera nos queda una sola opción para R3:

R3.1: Cas (3,2)

R3.2: Cas (3,4)

Fila 1: ■ □ ■ □

Fila 2: ■ ■ ■ ■

Fila 3: ■ ■ □ ■

Fila 4: □ □ ■ □

Por último, sabemos que como ya cumplimos con R5, Cas (4,1) no es posible nos queda una sola opción para R4

R4.1: Cas (4,2)

R4.2: Cas (4,4)

Fila 1: ■ □ ■ □

Fila 2: ■ ■ ■ ■

Fila 3: ■ ■ □ ■

Fila 4: □ ■ ■ ■

Construimos cumpliendo con R2, R7, R1, R5, R3 y R4. También podemos ver como gracias al cumplir con las mismas quedaron ya fijadas ciertas posibilidades para R6 y R8 las cuales son válidas, quedando como en lo anterior la tabla final.

Ejercicios de Implementación

6. Implementar un motor de inferencia con encadenamiento hacia adelante. Pruébalo con las proposiciones del ejercicio 3.

En el código mostrado en la figura comenzamos definiendo una clase Regla que tiene como atributos a las condiciones de una sentencia y su resultado. Dicha clase se adopta como atributo de la clase MotorInferencia así como los hechos actuales, y aclaro actuales porque al iniciar uno le entrega los iniciales pero a medida que se van demostrando sentencias los hechos que tenemos cambian.

Como métodos de la MotorInferencia tenemos: encadenamiento_adelante y mostrar_hechos, la segunda tiene un comportamiento evidente, printea todos los hechos encontrados, por lo que vamos a hacer hincapié en la segunda. El método encadenamiento_adelante comienza con una bandera que se encarga de verificar que no se hayan agregado nuevos hechos, ya que si no fue así significa que no hay más conclusiones que extraer con los hechos que tenemos. El while se encarga de evaluar el flag, luego recorre todas las reglas de la lista vector compuesta por objetos de la clase Regla con un iterador llamado regla, luego definimos dentro del for un vector de booleanos que se llena dentro de otro for que se encarga de recorrer las condiciones de regla con otro iterador llamado cond, el cual toma como valor cada una de las condiciones de la regla y evalúa si esa condición se encuentra en los hechos actuales. Cuando todas las condiciones de la regla son hechos (evaluando esto último con `all(vector de booleanos)`), sabemos que la conclusión es verdadera y la agregamos a los hechos, proceso que se repite hasta demostrar todo lo que se puede con los hechos iniciales y reglas que tengamos. También agregamos un sumador que se encarga de analizar en cuantas iteraciones obtuvimos la respuesta definitiva.

7. Implementar un motor de inferencia con encadenamiento hacia atrás. Pruébalo con las proposiciones del ejercicio 3.

1. Estructura general del programa

1. Definición de la base de reglas (reglas) — lista de tuplas (premisas, conclusión).
2. Inicialización: contador_iteraciones = 0 y hechos (conclusiones de reglas sin premisas).

3. Función principal de inferencia: `probar_objetivo(objetivo, visitados=None, nivel=0)` — realiza la búsqueda recursiva hacia atrás y muestra la traza.
4. Menú interactivo: `menu()` — permite ver la base, solicitar la demostración de una hipótesis y salir.

Bloque `if name == "main": menu()` — arranca el programa.

2. Representación de la BC

La base de conocimientos es una lista que tiene como elementos **tuplas**, estas tuplas tienen la forma (premisa, conclusion)

3. Inicialización de hechos

```
In [ ]: hechos = set([conclusion for premisas, conclusion in reglas if len(premisas) ==
```

- Esto crea un set con las conclusiones de reglas cuya lista de premisas está vacía.
- En tu base inicial hechos = {'d', 'e'} (R5 y R6).
- **Nota:** la comprensión desempaqueta cada tupla (premisas, conclusion) y por eso premisas aparece "sin haberse definido antes".

4. Funcionamiento de *probar_objetivo*

La función se comporta así (desglosado en pasos):

1. Incrementar contador (explicado arriba).
2. Inicializar visitados si es None: `visitados = set()`. Esto es el control de ciclo para la rama actual.
3. Impresión de traza: se usa nivel para producir una sangría `indent = " " * nivel` y luego `print(f"{indent}Intentando demostrar: {objetivo}")`.
4. Caso base rápido: si `objetivo in hechos` → imprime que es hecho conocido y devuelve True.
5. Detección de ciclos: si `objetivo in visitados` → ciclo detectado, imprime y devuelve False. Después se añade `visitados.add(objetivo)`.
6. Buscar reglas que concluyan el objetivo:
 - Se itera `for premisas, conclusion in reglas`: `if conclusion == objetivo`:
 - Si la regla tiene premisas imprime la forma $p1 \wedge p2 \rightarrow \text{objetivo}$, si no (hecho base) lo indica.
7. Probar todas las premisas:

- Se usa `all(probar_objetivo(p, visitados.copy(), nivel + 1) for p in premisas)`.
- `all(...)` llamará a `probar_objetivo` para cada `p` hasta que encuentre una premisa que falle (corta evaluación).
- Importante: se pasa `visitados.copy()` a cada premisa: cada rama de premisas recibe una copia del conjunto visitados, así cada sub-prueba no comparte nuevos marcados con las otras hermanas. Esto evita que una marca de una premisa afecte la evaluación de sus hermanas, pero también puede permitir que ciertas clases de ciclos entre premisas no se detecten. (Ver recomendaciones abajo.)

8. Si todas las premisas se demuestran:

- Se ejecuta `hechos.add(objetivo)` (memoización) y se imprime que se demostró, devuelve `True`.
9. Si ninguna regla pudo demostrar el objetivo: se imprime que no hay manera y devuelve `False`.

5. Conclusiones

El programa implementa correctamente un motor de encadenamiento hacia atrás con trazas por pantalla y un contador de llamadas. Su diseño es simple y efectivo para propósitos didácticos.

8. Implementar un motor de inferencia por contradicción que detecte si el conjunto de proposiciones del ejercicio 3 es inconsistente.

Introducción

El código implementa un motor de inferencia basado en resolución, un método de la lógica proposicional para demostrar proposiciones mediante prueba por contradicción.

La idea central es la siguiente:

1. Se convierte la base de conocimiento en Forma Normal Conjuntiva (FNC).
2. Se agrega la negación de la hipótesis a demostrar.
3. Se aplican reglas de resolución entre cláusulas hasta:
 - Encontrar la cláusula vacía (contradicción), lo que implica que la hipótesis es verdadera.
 - O bien, no poder generar nuevas cláusulas, en cuyo caso la hipótesis no es demostrable con la base de conocimiento actual.

Este procedimiento es uno de los más conocidos en sistemas de razonamiento automático y en el campo de la inteligencia artificial simbólica.

Conceptos clave

Literal

Un literal es la unidad más básica en lógica proposicional. Puede ser:

- Positivo: una proposición como a , b , c .
- Negativo: la negación de una proposición, escrita como $\neg a$, $\neg b$, $\neg c$.

Ejemplo: En la proposición $\neg p \vee q$, los literales son $\neg p$ y q .

Cláusula

Una cláusula es una disyunción (OR) de literales.

En el programa, cada cláusula se representa como un conjunto de literales en FNC.

Ejemplo: $\{\neg b, \neg c, a\}$ corresponde a la proposición $\neg b \vee \neg c \vee a$.

Forma Normal Conjuntiva (FNC)

La FNC es una forma estandarizada en lógica proposicional donde toda fórmula se representa como una conjunción (AND) de disyunciones (OR) de literales.

Ejemplo: $(\neg b \vee \neg c \vee a) \wedge (\neg d \vee \neg e \vee b) \wedge (\neg e \vee c)$

Resolvente

El resolvente es una nueva cláusula que se obtiene aplicando la regla de resolución a dos cláusulas que contienen un literal complementario (un literal y su negación).

Ejemplo:

Cláusula 1: $\{a, b\}$

Cláusula 2: $\{\neg a, c\}$

Resolvente: $\{b, c\}$ (porque a y $\neg a$ se eliminan).

Cláusula vacía

Si en el proceso de resolución se obtiene una cláusula vacía $\square$, significa que se encontró una contradicción. Esto prueba que la hipótesis es verdadera.

Estructura del código

El código define una clase principal `MotorInferenciaResolucion` que implementa todo el procedimiento. Sus componentes son:

1. Base de conocimiento (BC): Definida en el constructor **init**, con un conjunto de reglas y hechos expresados en FNC. Ejemplo: $b \wedge c \rightarrow a$ se convierte en $\neg b \vee \neg c \vee a$

- a.
2. Método `es_literal_complementario`: Comprueba si dos literales son opuestos (ejemplo: a y $\neg a$).
3. Método `aplicar_resolucion`: Genera un resolvente a partir de dos cláusulas con un único par de literales complementarios.
4. Métodos de visualización (`mostrar_clausula`, `mostrar_base_conocimiento`): Muestran de manera legible las cláusulas.
5. Método `demostrar_por_resolucion`: Implementa paso a paso la prueba de una hipótesis:
 - Muestra la base de conocimiento.
 - Agrega la negación de la hipótesis.
 - Aplica resolución iterativamente.
 - Si encuentra la cláusula vacía, concluye que la hipótesis es verdadera.
 - Si no, concluye que no puede demostrarse.
6. Método `demostrar_multiple`: Permite probar varias hipótesis y resumir los resultados.
7. Funciones `menu_interactivo` y `ejercicio_8`: Ofrecen un menú de interacción con el motor o la ejecución automática de la demostración de la hipótesis a .

Ejemplo en el código

La base de conocimiento inicial incluye reglas como:

- $b \wedge c \rightarrow a$ (equivalente en FNC a $\neg b \vee \neg c \vee a$)
- $d \wedge e \rightarrow b$ (equivalente a $\neg d \vee \neg e \vee b$)
- $e \rightarrow c$ (equivalente a $\neg e \vee c$)
- Hechos: d, e

Con estas reglas, el motor puede demostrar si la hipótesis a es verdadera. El sistema añade $\neg a$ y aplica la resolución paso a paso hasta llegar a la contradicción, concluyendo que a es demostrable.

Conclusión

El programa implementa un motor de inferencia por contradicción usando resolución, que permite demostrar proposiciones de manera automática.

Es un ejemplo práctico de cómo funcionan los sistemas expertos basados en lógica, un pilar de la inteligencia artificial simbólica. Con él se pueden:

- Definir una base de conocimiento en FNC.
- Probar hipótesis.
- Extender la base con nuevas cláusulas.
- Observar el proceso de resolución paso a paso.

Este motor es una herramienta didáctica valiosa para comprender la resolución en lógica proposicional y su aplicación en sistemas de inferencia.

**CONCLUSIÓN GLOBAL (comparando ejercicios 6, 7 y 8) EN EL PDF
ADJUNTO EN EL REPOSITORIO**

Bibliografía

Russell, S. & Norvig, P. (2004) *Inteligencia Artificial: Un Enfoque Moderno*. Pearson Educación S.A. (2a Ed.) Madrid, España

Poole, D. & Mackworth, A. (2023) *Artificial Intelligence: Foundations of Computational Agents*. Cambridge University Press (3a Ed.) Vancouver, Canada